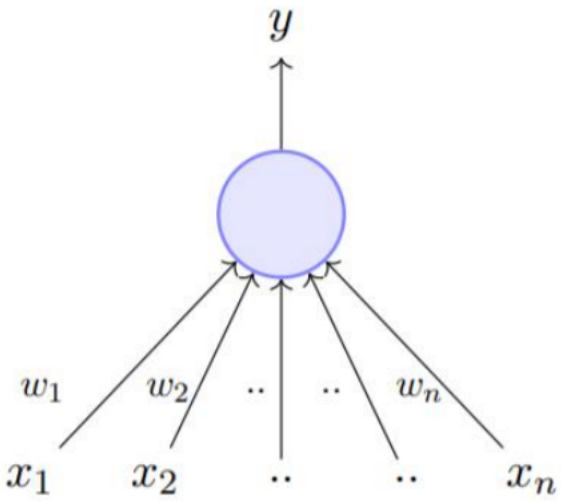
Perceptron and Adaline

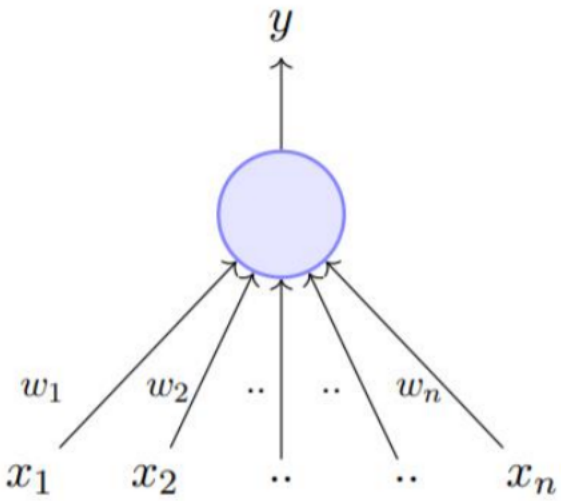
Xusiyuan
Siyuanxu9-c@my.cityu.edu.hk

Perceptron



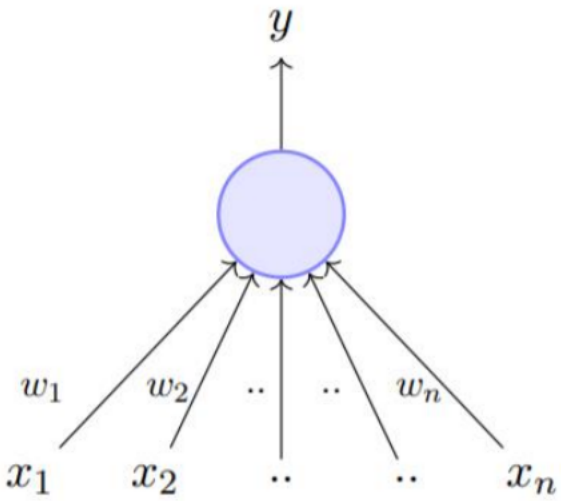
- Frank Rosenblatt, an American psychologist, proposed the classical perceptron model (1958)

Perceptron



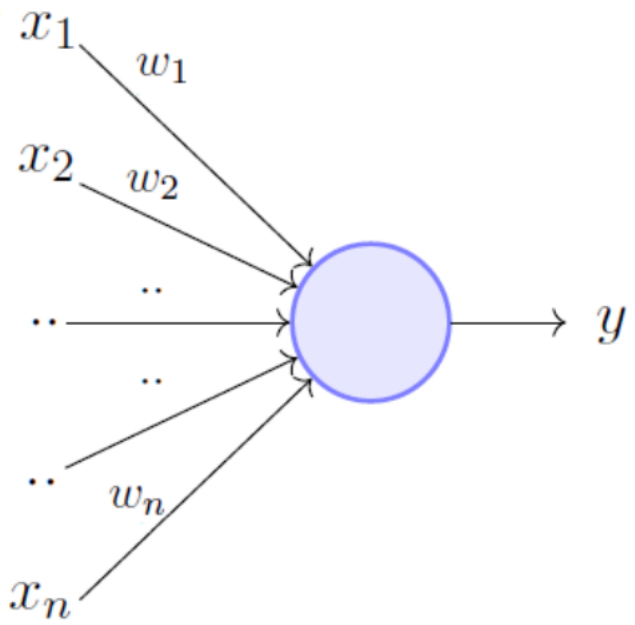
- Frank Rosenblatt, an American psychologist, proposed the classical perceptron model (1958)
- A more general computational model than McCulloch–Pitts neurons
- **Main differences**: Introduction of numerical weights for inputs and a mechanism for learning these weights
- Inputs are no longer limited to boolean values

Perceptron



- Frank Rosenblatt, an American psychologist, proposed the **classical perceptron** model (1958)
- A more general computational model than McCulloch–Pitts neurons
- **Main differences**: Introduction of numerical weights for inputs and a mechanism for learning these weights
- Inputs are no longer limited to boolean values
- Refined and carefully analyzed by Minsky and Papert (1969) - their model is referred to as the **perceptron** model here

Perceptron



$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i \geq \theta$$

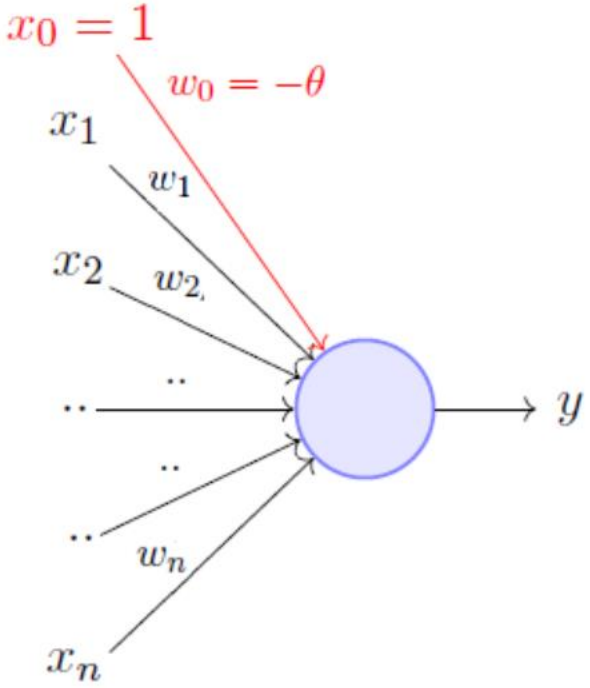
$$= 0 \quad if \sum_{i=1}^{n} w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i - \theta \geq 0$$

$$= 0 \quad if \sum_{i=1}^{n} w_i * x_i - \theta < 0$$

Perceptron



A more accepted convention,

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

$$where, \quad x_0 = 1 \quad and \quad w_0 = -\theta$$

whether to buy a game on steam or not

The **inputs** are all boolean i.e., {0,1}

Output variable is also boolean {0: Will buy it, 1: Won't buy it}.

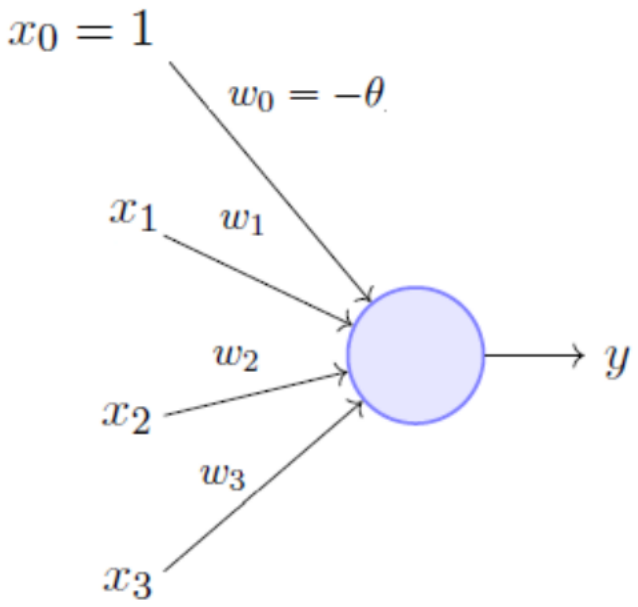
x_1: Is there a Chinese language interface?

x_2: Are reviews positive ?

x_3: Is there a discount?



Perceptron



x_1: Is there a Chinese language interface?

x_2: Are reviews positive ?

x_3: Is there a discount?

Perceptron vs McCulloch-Pitts Neuron

McCulloch Pitts Neuron
(assuming no inhibitory inputs)

$$y = 1 \quad if \sum_{i=0}^{n} x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} x_i < 0$$

Perceptron

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

Perceptron

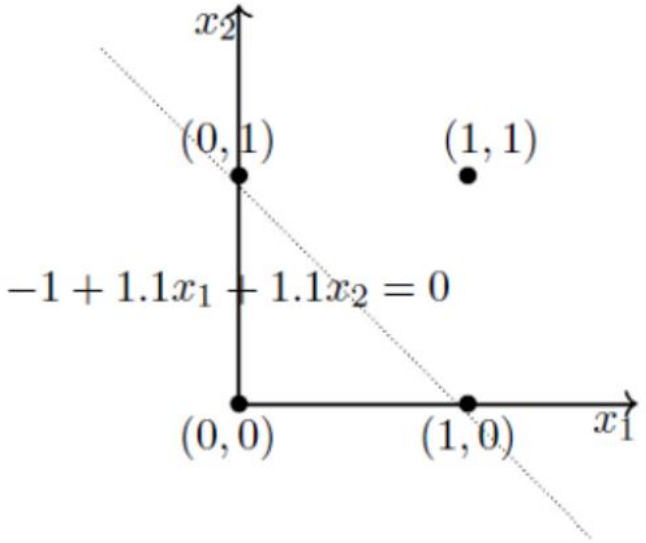
- **OR Function**

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 > -w_0$$

One possible solution is

$w_0 = -1,\ w_1 = 1.1,\ w_2 = 1.1$



Perceptron

- **XOR Function**

Truth Table

Input		Output
A	B	A **XOR** B
0	0	0
0	1	1
1	0	1
1	1	0

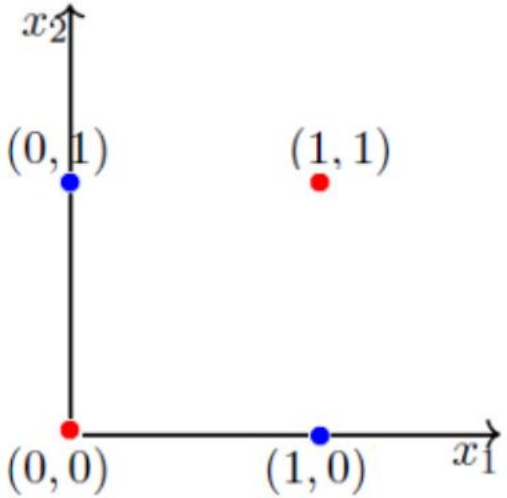
- If the inputs are the same, then the output of an XOR gate is false
- If the inputs are different, then the output of an XOR gate is true

Perceptron

- **XOR Function**

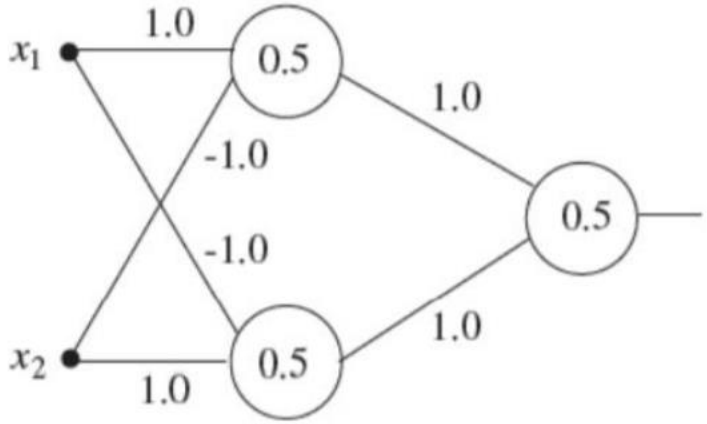
x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^{2} w_i x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^{2} w_i x_i < 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$
$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 < -w_0$$



Multi-layer Perceptron

- XOR Function



Adaline

- Adaptive Linear Neuron
- A weighted sum with a bias
- Learning uses the continuous output before thresholding

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

Class label: +1 if $z \geq 0$, otherwise -1

Learning Rule

Error: $e = t - z$

Loss for one example: $L = (t - z)^2$

$w_i(new) = w_i(old) + \eta\, e\, x_i$

$b(new) = b(old) + \eta\, e$

η: learning rate. Repeat updates over the training data.

One Training Update

$x = (1, 2),\ t = +1,\ \eta = 0.1$

Before: $w = (0.1, -0.1),\ b = 0$

$z = 0.1 \times 1 - 0.1 \times 2 = -0.1$

$e = 1 - (-0.1) = 1.1$



$w_1(\text{new}) = 0.1 + 0.1 \times 1.1 \times 1 = 0.21$

$w_2(\text{new}) = -0.1 + 0.1 \times 1.1 \times 2 = 0.12$

$b(\text{new}) = 0 + 0.1 \times 1.1 = 0.11$

After: w = (0.21, 0.12), b = 0.11

New score: 0.56 Loss: 0.605 to 0.0968

References

https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1

https://www.youtube.com/watch?v=6XhSJbfT1pk

https://www.cse.iitm.ac.in/~miteshk/CS7015.html

https://www.cs.cmu.edu/~tom/10601_fall2012/recitations/Recitation_2Oct2012_PG.pdf

https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent

Thanks

Xusiyuan